

Universidad San Carlos de Guatemala

Facultad de ingeniería.

Ingeniería en ciencias y sistemas



Título del Proyecto:

EDD CinemaUsac Fase 3

PONDERACIÓN: 50

Horas Aproximadas: 35

Índice

1. Resumen Ejecutivo	4
2. Competencia que desarrollaremos	4
3. Objetivos del Aprendizaje	5
3.1 Objetivo General	5
3.2 Objetivos Específicos	5
4. Enunciado del Proyecto	6
4.1 Descripción del problema a resolver	6
1. Gestión de Grafo de Relaciones del Sistema	6
2. Gestión del Blockchain de Transacciones	7
3. Verificación del Árbol de Merkle	7
4. Compresión y Generación de Archivos LZW	7
5. Visualización de Relaciones del Sistema (Grafo Interactivo)	8
6. Auditoría de Integridad de Datos (Blockchain + Merkle)	8
1. Registro de Usuarios	8
2. Inicio de Sesión	8
3. Consulta de Cartelera	9
4. Reserva de Asientos	9
5. Cancelación de Reservas	9
6. Consulta de Promociones	9
7. Solicitudes Especiales	9
8. Estado de Solicitudes	9
Área de Reportes (Fase 3 – CinemaUSAC)	10
Reporte 1: Grafo No Dirigido del Sistema (Lista de Adyacencia)	10
Reporte 2: Blockchain de Transacciones	10
Reporte 3: Árbol de Merkle (Integridad de Datos)	11
Reporte 4: Archivos de Compresión LZW (.lzw)	11
Reporte 5: Grafo de Relaciones de Funciones y Asientos	12
Seguridad	12
Estructura de un bloque(json)	14
Almacenamiento de bloques	14
Árbol de Merkle	14
Protección de datos	16
4.2 Alcance del proyecto	16
5. Desarrollo de Habilidades Blandas	17
5.1 Proyectos Individuales	17
5.2 Resumen de Puntuaciones	18
5.3 Observaciones	19
5.4 Comentarios Generales	20

1. Resumen Ejecutivo

El sistema CinemaUSAC evoluciona hacia una tercera fase orientada a la implementación de estructuras avanzadas para el manejo eficiente, seguro y distribuido de la información del complejo de cines. En esta etapa, se incorpora el uso de un grafo no dirigido basado en lista de adyacencia para modelar relaciones entre entidades del sistema como salas, usuarios, funciones y rutas operativas, permitiendo analizar conexiones y optimizar la interacción entre componentes del cine.

Adicionalmente, se integra la tecnología de blockchain para garantizar la integridad y trazabilidad de las transacciones del sistema, especialmente en procesos críticos como compras, reservas y movimientos de inventario. Dentro de esta misma línea de seguridad, se implementa el árbol de Merkle, utilizado para la verificación eficiente e inmutable de bloques de datos, asegurando la consistencia de la información almacenada.

Finalmente, se incorpora un sistema de compresión LZW (Lempel-Ziv-Welch) para la optimización del almacenamiento y transferencia de datos, generando archivos con extensión .lzw, donde la información será representada en formato hexadecimal que codifica los bytes de cada "code word". Esto permitirá reducir significativamente el tamaño de los archivos generados por el sistema, mejorando el rendimiento y eficiencia en el manejo de grandes volúmenes de información.

2. Competencia que desarrollaremos

A través de esta fase del proyecto CinemaUSAC, el estudiante fortalecerá competencias avanzadas en el diseño e implementación de estructuras de datos no lineales y sistemas de gestión distribuidos, aplicados a un entorno real de procesamiento de información compleja.

Se desarrollará la capacidad de modelar problemas mediante grafos no dirigidos con listas de adyacencia, comprendiendo su aplicación en la representación de relaciones dinámicas entre entidades del sistema. Asimismo, se adquirirán habilidades en el diseño e implementación de sistemas basados en blockchain, entendiendo los principios de inmutabilidad, descentralización y verificación de datos.

El estudiante también consolidará conocimientos en estructuras criptográficas como el árbol de Merkle, aplicándolo en la validación eficiente de grandes volúmenes de información, asegurando integridad y consistencia de datos en entornos distribuidos.

De igual forma, se fortalecerán competencias en algoritmos de compresión de datos mediante la implementación del método LZW, desarrollando la capacidad de optimizar almacenamiento y representar información en formatos compactos como archivos .lzw en hexadecimal. Todo esto se complementa con habilidades de programación algorítmica avanzada, pensamiento analítico, y diseño de sistemas robustos orientados a eficiencia, seguridad y escalabilidad.

3. Objetivos del Aprendizaje

3.1 Objetivo General

Desarrollar un sistema avanzado de gestión para el complejo de cines CinemaUSAC que implemente estructuras de datos no lineales y tecnologías de integridad y compresión de información, tales como grafos no dirigidos, blockchain, árboles de Merkle y algoritmos de compresión LZW, aplicados al manejo eficiente, seguro y optimizado de datos operativos.

3.2 Objetivos Específicos

Al finalizar el proyecto, los estudiantes deberán ser capaces de:

- Implementar un grafo no dirigido mediante lista de adyacencia para modelar y analizar las relaciones entre los distintos componentes del sistema CinemaUSAC, como usuarios, salas, funciones y procesos operativos.
- Desarrollar un sistema de blockchain funcional que permita garantizar la trazabilidad, seguridad e inmutabilidad de las transacciones realizadas dentro del sistema de gestión del cine.
- Aplicar la estructura de árbol de Merkle para la verificación eficiente de la integridad de datos, asegurando la consistencia de bloques de información en el sistema.
- Implementar el algoritmo de compresión LZW, generando archivos con extensión .lzw que almacenen información en formato hexadecimal representando los bytes de cada code word, optimizando el uso de almacenamiento y transferencia de datos.

4. Enunciado del Proyecto

4.1 Descripción del problema a resolver

Funciones de Administrador

El administrador del sistema contará con un menú principal en consola que le permitirá realizar las siguientes funciones:

1. Gestión de Grafo de Relaciones del Sistema

El administrador podrá visualizar y administrar un **grafo no dirigido implementado mediante lista de adyacencia**, donde se representan las relaciones entre entidades del cine como películas, funciones, salas, usuarios y promociones. Cada nodo representa una entidad y cada arista una relación operativa (por ejemplo: *película* → *función*, *función* → *sala*, *usuario* → *reserva*).

El sistema permitirá agregar nodos y relaciones dinámicamente, así como consultar conexiones directas e indirectas. Esto facilita el análisis de interacción entre componentes del sistema y permite detectar dependencias operativas dentro del cine.

```
1  {
2    "películas": [
3      {
4        "codigo": "P001",
5        "titulo": "Avengers: Endgame",
6        "genero": "Acción",
7        "duracion": 181,
8        "clasificacion": "B15",
9        "idioma": "Doblada",
10       "estreno": "2026-05-01",
11       "fin_cartelera": "2026-05-20"
12     },
13     {
14       "codigo": "P002",
15       "titulo": "Toy Story 5",
16       "genero": "Animación",
17       "duracion": 110,
18       "clasificacion": "A",
19       "idioma": "Español",
20       "estreno": "2026-05-10",
21       "fin_cartelera": "2026-06-01"
22     }
23   ],
24   "transacciones_blockchain": [
25     {
26       "id": "T001",
27       "tipo": "RESERVA",
28       "usuario": "Juan Perez",
29       "pelicula": "P001",
30       "asiento": "F5-10",
31       "timestamp": "2026-05-10T14:30:00"
32     },
33     {
34       "id": "T002",
35       "tipo": "COMPRA",
36       "usuario": "Maria Lopez",
37       "producto": "Combo Familiar",
38       "monto": 75.50,
39       "timestamp": "2026-05-10T15:00:00"
40     }
41   ],
42 }
```

```

42     "promociones": [
43     {
44         "codigo": "PR01",
45         "nombre": "Martes de Locura",
46         "vigencia": {
47             "inicio": "2026-05-01",
48             "fin": "2026-06-01"
49         },
50         "dias": [
51             "Martes"
52         ],
53         "beneficios": [
54             {
55                 "tipo": "descuento",
56                 "descripcion": "50% en entradas",
57                 "valor": 50
58             }
59         ]
60     }
61 ]
62 }

```

2. Gestión del Blockchain de Transacciones

El administrador podrá visualizar el **blockchain interno del sistema**, el cual registra todas las transacciones críticas como reservas, cancelaciones, compras y movimientos de inventario. Cada bloque contiene un hash, timestamp, datos de la transacción y el hash del bloque anterior.

El sistema validará la integridad de la cadena en tiempo real, permitiendo detectar cualquier alteración en los datos. Esta estructura garantiza seguridad, trazabilidad e inmutabilidad de las operaciones del sistema CinemaUSAC.

3. Verificación del Árbol de Merkle

El administrador podrá generar y visualizar el **árbol de Merkle** asociado a bloques de transacciones o conjuntos de datos del sistema. Cada hoja del árbol representa una transacción o registro, mientras que los nodos internos contienen hashes combinados.

Esta estructura permite validar rápidamente la integridad de grandes volúmenes de información sin necesidad de revisar todos los datos individualmente, optimizando procesos de auditoría y verificación.

4. Compresión y Generación de Archivos LZW

El administrador podrá ejecutar el proceso de **compresión LZW** sobre archivos generados por el sistema (reportes de reservas, transacciones o inventario). El resultado será un archivo con extensión **.lzw**, donde los datos estarán representados en formato hexadecimal correspondiente a los code words generados por el algoritmo.

Esto permite reducir significativamente el tamaño de almacenamiento y mejorar la eficiencia en la transferencia de datos dentro del sistema.

5. Visualización de Relaciones del Sistema (Grafo Interactivo)

El administrador podrá visualizar el grafo completo del sistema en consola o mediante exportación, mostrando nodos y conexiones entre entidades. Esta funcionalidad permite analizar el comportamiento del sistema, rutas de interacción y dependencias entre módulos.

El sistema permitirá recorrer el grafo, buscar conexiones específicas y exportar la estructura para análisis externo.

6. Auditoría de Integridad de Datos (Blockchain + Merkle)

El administrador podrá ejecutar una auditoría completa del sistema, donde se valida simultáneamente:

- La integridad del **blockchain**
- La consistencia del **árbol de Merkle**
- La coherencia de datos entre nodos del grafo

El sistema generará un reporte indicando si los datos son válidos o si existe alguna alteración, permitiendo asegurar la confiabilidad total del sistema CinemaUSAC.

Funciones de Cliente en Taquilla

Los usuarios tendrán acceso a las siguientes funciones:

1. Registro de Usuarios

El cliente podrá registrarse mediante un formulario gráfico donde se almacenan los datos en una **lista enlazada simple**. Cada nodo representa un usuario con información básica como DPI, nombre, teléfono y contraseña.

Esto permite mantener un registro dinámico de usuarios sin necesidad de estructuras complejas.

2. Inicio de Sesión

Los usuarios registrados podrán autenticarse mediante DPI y contraseña. El sistema validará los datos recorriendo la lista enlazada simple de usuarios.

Si la autenticación es correcta, se habilita el acceso al menú principal del cliente.

3. Consulta de Cartelera

El cliente podrá visualizar todas las películas disponibles, almacenadas en una **lista doblemente enlazada ordenada por código**.

También podrá filtrar por género, título o clasificación, y ver detalles como duración, idioma y horarios disponibles.

4. Reserva de Asientos

El cliente podrá seleccionar una película, horario y sala. El sistema mostrará una **matriz dispersa que representa los asientos** de la función.

- Si el asiento está libre → se reserva
- Si está ocupado → se notifica y se recomienda otro asiento

La matriz dispersa se actualiza dinámicamente con el nombre del cliente en la posición seleccionada.

5. Cancelación de Reservas

El cliente podrá cancelar reservas activas. El sistema buscará en la matriz dispersa los asientos asociados al usuario y los liberará.

Esto se refleja en tiempo real, cambiando el estado del asiento a **LIBRE** dentro de la estructura.

6. Consulta de Promociones

El cliente podrá visualizar promociones activas recorriendo la **lista circular doble de promociones**, mostrando beneficios, vigencia y días aplicables.

7. Solicitudes Especiales

El cliente podrá enviar solicitudes (cumpleaños, eventos, quejas). Estas se insertan en una **lista circular doblemente enlazada**, con prioridad y estado.

8. Estado de Solicitudes

El cliente podrá consultar el estado de sus solicitudes ingresando su teléfono. El sistema recorre la lista circular y muestra:

- Número de solicitud
- Tipo
- Descripción

- Estado
- Fecha

Área de Reportes (Fase 3 – CinemaUSAC)

El sistema contará con un módulo avanzado de generación de reportes, enfocado en la visualización de estructuras de datos en tiempo real, utilizando Graphviz y archivos de salida especializados. Estos reportes permiten auditar, analizar y comprender el comportamiento interno del sistema CinemaUSAC desde una perspectiva gráfica y estructural.

Reporte 1: Grafo No Dirigido del Sistema (Lista de Adyacencia)

Este reporte muestra la representación completa del sistema mediante un **grafo no dirigido implementado con lista de adyacencia**, donde cada nodo representa una entidad del sistema como usuarios, películas, funciones, salas, promociones y transacciones.

Cada nodo se conecta con otros según su relación operativa, por ejemplo:

- Usuario → Reserva
- Película → Función
- Función → Sala
- Promoción → Beneficio

Características:

- Nodos representados como círculos o rectángulos
- Aristas bidireccionales (grafo no dirigido)
- Lista de adyacencia como base lógica del grafo
- Colores por tipo de entidad (usuarios, películas, transacciones)
- Permite visualizar conexiones directas e indirectas
- Exportación en Graphviz (.dot)

Reporte 2: Blockchain de Transacciones

Este reporte muestra la cadena de bloques generada por el sistema, donde cada bloque representa una transacción (reservas, compras, cancelaciones o movimientos de inventario).

Cada bloque contiene:

- ID del bloque
- Hash actual
- Hash del bloque anterior
- Datos de la transacción

- Timestamp

Características:

- Estructura lineal encadenada
- Validación de integridad entre bloques
- Visualización del hash linking
- Detección de alteraciones (si un bloque cambia, se rompe la cadena)
- Representación en Graphviz tipo flujo vertical

Reporte 3: Árbol de Merkle (Integridad de Datos)

Este reporte visualiza el **árbol de Merkle**, utilizado para validar la integridad de transacciones del sistema sin necesidad de revisar cada registro individual.

Cada hoja representa una transacción y los nodos internos contienen hashes combinados.

Características:

- Estructura jerárquica tipo árbol binario
- Hojas = transacciones
- Nodos internos = hash combinado
- Raíz del árbol = hash global de integridad
- Validación rápida de cambios en datos
- Visualización eficiente para auditoría

Reporte 4: Archivos de Compresión LZW (.lzw)

Este reporte muestra los archivos generados mediante el algoritmo de **compresión LZW**, utilizados para optimizar almacenamiento del sistema.

El contenido del archivo se representa en:

- Secuencias de **code words**
- Representación en **formato hexadecimal**
- Relación entre entrada original y salida comprimida

Características:

- Visualización del flujo de compresión LZW
- Tabla de códigos generados
- Representación hexadecimal de bytes
- Comparación tamaño original vs comprimido
- Exportación de archivos **.lzw**

Reporte 5: Grafo de Relaciones de Funciones y Asientos

Este reporte muestra un **grafo no dirigido especializado** que conecta películas, funciones, salas y asientos.

Ejemplo de relaciones:

- Película → Función
- Función → Sala
- Sala → Asientos VIP / normales / accesibles
- Usuario → Asiento reservado

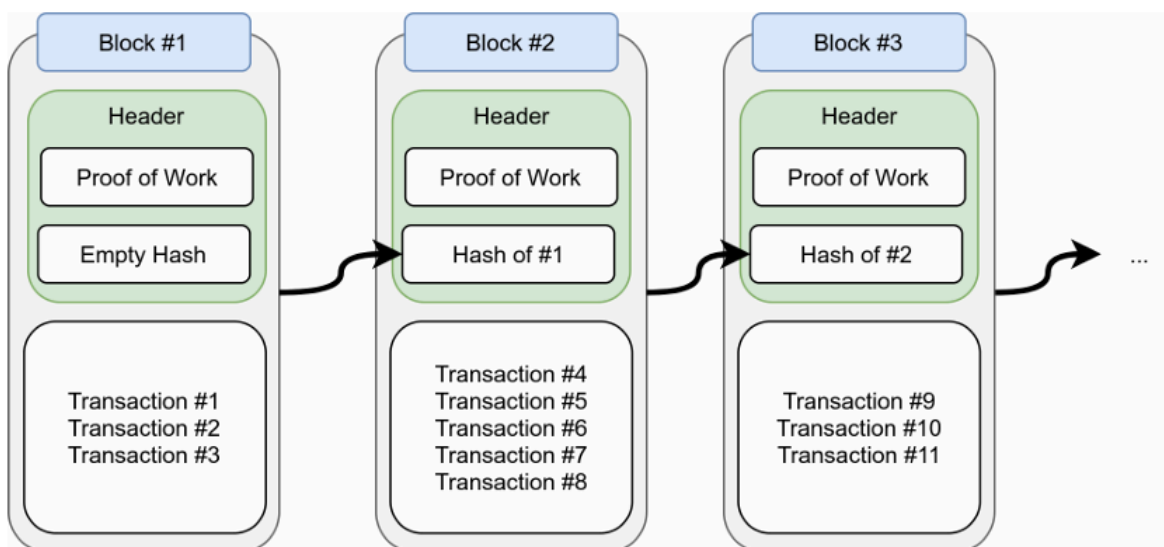
Características:

- Lista de adyacencia como base estructural
- Nodos clasificados por entidad operativa
- Visualización de disponibilidad de asientos
- Colores por tipo de asiento (VIP, libre, ocupado, accesible)
- Análisis de ocupación por función

Seguridad

Blockchain

Esta es una estructura de bloques, los cuales se comportan como una lista enlazada simple, en donde cada uno de los nodos(bloques) se ingresan por delante.



Prueba de trabajo: Es el proceso por el cual se encuentra un hash que cumpla con la condición de tener un prefijo de n ceros, donde n corresponde a un número entero, para el

caso de la aplicación este poseerá un valor de 4. Para esto se debe iterar un número entero (NONCE) el cual iniciará desde cero y se incrementará en 1 hasta encontrar un hash válido para el bloque.

Generación de nuevos bloques: Un nuevo bloque se deberá generar cada vez que se cree una publicación o un comentario, almacenando en el campo DATA todas las publicaciones existentes con sus comentarios.

Bloque

El bloque se compone de distintas partes que contienen información acerca del mismo, estas partes son:

- INDEX: Representa el número del bloque que se estará insertando en la cadena, este campo es incrementado en uno para cada bloque que se inserta por lo cual, naturalmente, el primer bloque tendrá un índice de cero y los demás tendrán índices con valores sucesivos (1,2,3,etc.).
- TIMESTAMP: Tendrá la fecha y hora en la cual se creó el bloque, poseerá el siguiente formato: (DD-MM-YY::HH:MM:SS).
- DATA: Este campo poseerá toda la información de las publicaciones(lista doble) y comentarios.
- NONCE: Será el número entero que se debe iterar de uno en uno hasta encontrar un hash que cumpla con la prueba de trabajo, es decir, que contenga el prefijo con la cantidad de 0s designada (cuatro).
- PREVIOUSHASH: Corresponde al hash del bloque anterior a cada uno de los bloques. Esto actúa como un apuntador que indica si los datos en algún punto de la cadena de bloques fueron corrompidos, en el caso del primer bloque no se posee ningún bloque previo por lo cual el valor de este apuntador será "0000".
- HASH: El hash que identifica al bloque actual y protege la data que este almacena de ser corrompida. Este hash se genera aplicando la función SHA256 a las propiedades: INDEX, TIMESTAMP, NONCE, ROOTMERKLE y PREVIOUSHASH. Todas deben ser cadenas concatenadas sin espacios en blanco ni saltos de línea.

SHA256(INDEX + TIMESTAMP+ NONCE + ROOTMERKLE+ PREVIOUSHASH)

Estructura de un bloque(json)

```
{
  "INDEX": 0,
  "TIMESTAMP": "21-09-2024::11:30:29",
  "NONCE": 114706,
  "DATA": [{
    "correo": "maria@gmail.com",
    "contenido": "Holaaaaaa",
    "fecha": "19/08/2024",
    "hora": "10:00",
    "comentarios": [{
      "correo": "pedro@gmail.com",
      "comentario": "Como te va",
      "fecha": "20/08/2024",
      "hora": "11:00"
    }, {
      "correo": "pedro@gmail.com",
      "comentario": "Como te va",
      "fecha": "20/08/2024",
      "hora": "11:00"
    }]
  }, {
    "correo": "jose@gmail.com",
    "contenido": "Tengo hambre, que recomiendan?",
    "fecha": "19/08/2024",
    "hora": "10:00",
    "comentarios": []
  }],
  "PREVIOUSHASH": "0000",
  "HASH": "00003d3b856ea2d3ffdd32bd820e7448acf217974b752b469b4bdce206767c7e"
}
```

Almacenamiento de bloques

Los bloques deben ser almacenados en archivos JSON, para que estos puedan ser leídos al iniciar la aplicación y poder tener persistencia de la información de las publicaciones. Para esto se deberá de tener una carpeta en la raíz de su proyecto que llevará por nombre blockchain. En esta carpeta se deberá almacenar todos los bloques que se generen durante la ejecución de la aplicación. Estos bloques deberán ser leídos al momento de iniciar la aplicación.

Árbol de Merkle

Un árbol de Merkle es una estructura de datos en forma de árbol binario en el que cada nodo hoja contiene un valor hash de un bloque de datos, y cada nodo intermedio contiene el hash combinado de sus nodos hijos. Al final, el nodo raíz del árbol (conocido como Merkle Root) representa un resumen criptográfico de todo el conjunto de datos.

Por lo tanto, el administrador tendrá la opción de generar el árbol del último bloque agregado en el blockchain, donde cada publicación con sus comentarios del campo DATA del bloque será un nodo hijo del árbol de Merkle. Tomando como referencia la estructura de un bloque, el árbol estaría compuesto por nodos hoja correspondientes a cada elemento de información, por ejemplo, uno para Maria y otro para José.

Ejemplo:

hash = funcionHash(Data)

Hash: será el identificador correspondiente a la data que se almacenará en los nodos hoja del árbol de Merkle.

FunciónHash: es el método utilizado para encriptar la información que servirá de entrada para la función. Para este caso se utilizará SHA256.

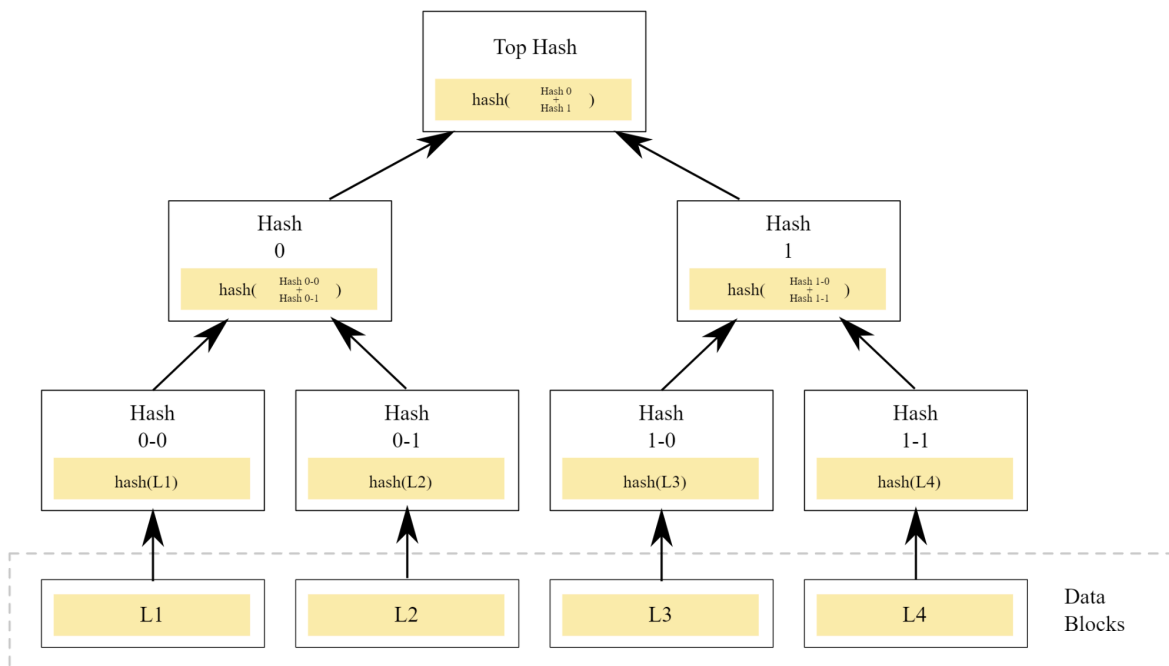
Data: es la publicación con sus comentarios.

Ejemplos de nodos hoja:

- hash = sha256(DATA)
- hash = 39b923082a194166d8d92989116bd

Ejemplos de nodos internos:

- hash = sha256("39b923082a194166d8d92989116bd" + "aa17d5fdf24761b54ad640691146656095b")
- hash = 113b685b12dbfa76c04bec7759a24ba66dd



Protección de datos

Desde el inicio de la aplicación se ha almacenado la información sensible de los usuarios, como contraseñas, de forma no segura, por lo cual es necesario corregir esta debilidad e implementar buenas prácticas de seguridad para salvaguardar dicha información.

Para lograr este objetivo, se debe hacer uso de la función hash criptográfica SHA256 en las contraseñas de los usuarios al momento de realizar una carga masiva, registrar un usuario o cuando el usuario edite su contraseña.

Además de implementar medidas de seguridad en los datos sensibles de los usuarios, se debe asegurar que los datos de los mismos (datos de usuario y amigos) no sean modificados desde el archivo JSON en el cual se están almacenando. Es por esto que la Blockchain debe ser capaz de identificar esta manipulación en los datos y debe ser representada en el grafo de la misma.

Esta manipulación en los datos, en caso de existir, deberá representarse coloreando el nodo de la blockchain que fue manipulado de color rojo. Si por el contrario no existe manipulación, el nodo deberá colorearse de color verde.

The image displays three screenshots of a blockchain interface, each representing a different block. Block 1 is highlighted in green, while Blocks 2 and 3 are highlighted in red. Each block contains a list of transactions, a previous hash, and a new hash. The transactions are represented as JSON objects with fields for sender, content, date, time, and comments. The previous hash is a long hexadecimal string, and the new hash is a shorter hexadecimal string. A 'Mine' button is visible at the bottom of each block.

Block	Nonce	Prev	Hash
1	76399	00	00
2	11379	00	00
3	41502	284625e0452c96b2370a32fe4f9818d07577ad4	d791daead72242d78b48d28cfeb3ec867cc311

4.2 Alcance del proyecto

Las estructuras de datos correspondientes a la tercera fase del sistema CinemaUSAC deben ser desarrolladas completamente por los estudiantes, sin utilizar librerías o módulos predefinidos en Perl o cualquier otro lenguaje que implementen directamente grafos, blockchain, árboles de Merkle o compresión LZW. Toda la lógica de construcción, enlace, inserción, recorrido, validación y manipulación debe ser implementada de forma manual.

En esta fase, el proyecto se enfoca en estructuras de mayor complejidad y seguridad, por lo que el estudiante debe garantizar la correcta implementación de:

- Grafos no dirigidos mediante listas de adyacencia
- Blockchain para registro e integridad de transacciones
- Árbol de Merkle para verificación criptográfica de datos
- Algoritmo de compresión LZW con salida en archivos .lzw en formato hexadecimal

Los reportes gráficos son un componente crítico de evaluación. Cada estructura debe contar con su respectiva representación en Graphviz. La ausencia de cualquier reporte gráfico implicará la anulación de los puntos asociados a dicha estructura, sin excepción.

El código fuente debe ser completamente original, desarrollado de forma individual, debidamente estructurado en módulos y con comentarios explicativos que reflejen la comprensión del funcionamiento de cada estructura implementada.

4.2 Requerimiento técnicos

Implementación del sistema en Perl

El sistema se desarrollará completamente en Perl mediante una aplicación de consola que gestione todas las funcionalidades de la Fase 3. Se hará uso de las capacidades nativas del lenguaje para el manejo de archivos, procesamiento de texto y generación de estructuras complejas.

El sistema deberá incluir módulos claramente definidos para cada estructura:

- Módulo de grafos no dirigidos (lista de adyacencia)
- Módulo de blockchain
- Módulo de árbol de Merkle
- Módulo de compresión LZW
- Módulo de generación de reportes

Cada módulo debe ser independiente, reutilizable y correctamente integrado dentro del sistema principal.

Generación de reportes con Graphviz

Todos los reportes gráficos deberán generarse exclusivamente utilizando Graphviz. El sistema en Perl será responsable de crear archivos en formato DOT que describan cada estructura de datos.

Posteriormente, estos archivos serán procesados para generar representaciones visuales en formato PNG o PDF.

Los reportes requeridos incluyen:

- Grafo del sistema (lista de adyacencia)
- Visualización del blockchain
- Árbol de Merkle del último bloque
- Estructura de compresión LZW
- Relación entre entidades del sistema

Los reportes deberán actualizarse automáticamente cada vez que se realice una modificación en las estructuras de datos, garantizando una visualización en tiempo real del estado del sistema.

5. Desarrollo de Habilidades Blandas

En la tercera fase del proyecto, el enfoque en habilidades blandas se mantiene, pero se orienta hacia el fortalecimiento de capacidades avanzadas relacionadas con el desarrollo de sistemas complejos, seguridad de la información y gestión de estructuras distribuidas.

Estas habilidades complementan el componente técnico del proyecto, preparando al estudiante para entornos profesionales donde la toma de decisiones, el análisis crítico y la responsabilidad individual son fundamentales.

5.1 Proyectos Individuales

Los proyectos individuales en esta fase permiten al estudiante asumir la responsabilidad total del diseño e implementación de un sistema complejo basado en estructuras avanzadas de datos y seguridad informática.

El estudiante debe gestionar de forma autónoma todas las etapas del desarrollo, desde el análisis hasta la implementación de algoritmos de grafos, blockchain, árboles de Merkle y compresión LZW.

5.1.1 Autogestión del Tiempo

El estudiante deberá planificar de manera estructurada el desarrollo del sistema, dividiendo el trabajo en módulos correspondientes a cada estructura de la Fase 3. Esto implica establecer cronogramas realistas para la implementación de grafos, blockchain, Merkle Tree y LZW, asegurando el cumplimiento de entregas parciales y finales.

5.1.2 Responsabilidad y Compromiso

En esta fase, el estudiante es responsable de la correcta implementación de estructuras avanzadas que requieren precisión algorítmica. Esto incluye garantizar la integridad del blockchain, la correcta generación del árbol de Merkle y la compresión adecuada de datos mediante LZW.

El compromiso implica entregar un sistema funcional, estable y correctamente documentado.

5.1.3 Resolución de Problemas

La complejidad de la Fase 3 exige que el estudiante enfrente problemas relacionados con seguridad, eficiencia y consistencia de datos. Será necesario resolver errores en la implementación de grafos, validar hashes en blockchain, optimizar el árbol de Merkle y corregir posibles pérdidas de información en la compresión LZW.

Esto fortalece la capacidad de análisis y depuración de sistemas complejos.

5.1.4 Reflexión Personal

Al finalizar la Fase 3, el estudiante deberá realizar una autoevaluación crítica sobre su desempeño en la implementación de estructuras avanzadas. Esto incluye analizar la eficiencia de su diseño, la calidad de su código, la seguridad del sistema y las mejoras posibles en cada estructura implementada.

Esta reflexión fomenta el aprendizaje continuo y la mejora en el desarrollo de sistemas de software de alta complejidad.

5.2 Resumen de Puntuaciones

Área	Puntos Totales	Puntos Obtenidos
1. Implementación de estructuras de datos		
Grafo no dirigido	7	
Lista de adyacencia	8	
Blockchain	3	
árbol merkle	7	
compresion lzw	15	
Sub-Total	40	

2. Funcionalidades del sistema		
Funciones de administrador	18	
Funciones de usuario de taquilla	12	
Sub-Total	30	
3. Reporte gráficos con Graphviz		
Reporte de grafo no dirigido	4	
Reporte de lista de adyacencia	4	
Reporte de árbol merkle	4	
Reporte blockchain	8	
Sub-Total	20	
4. Manual técnico y documentación	10	
TOTAL	100	

5.3 Observaciones

- El lenguaje de programación para esta fase será Perl.
- La herramienta de desarrollo de reportes gráficos será exclusivamente Graphviz.
- La entrega se realizará por medio de Github. El nombre del repositorio debe ser EDD_2S2026_PY_#carnet (Ej: EDD_1S2026_PY_202102771), donde se creará una carpeta con el nombre EDD_Proyecto1_Fase1.
- Por medio de UEDI se hará entrega del link del repositorio.
- Deberá agregar a su respectivo auxiliar a su reporte como colaborador:
 - jenzzzjs → Sección B
 - @seb4sr → Sección C
 - Ggio21 → sección A
- Los repositorios deben mantenerse en privado para evitar copias de código entre estudiantes.
- Se debe realizar el manual técnico en formato Markdown dentro de una carpeta llamada documentación, dentro de la carpeta de la respectiva fase, explicando la implementación de cada estructura y las decisiones de diseño tomadas.
- Todas las dudas durante el proceso de desarrollo deberán realizarse por medio de los foros de UEDI de la respectiva sección.
- Las copias detectadas serán penalizadas con calificación de 0 puntos y reportadas a la Escuela de Ciencias y Sistemas.
- Se debe de realizar commits frecuentes en Github para demostrar el avance progresivo del proyecto.
- El código debe estar debidamente comentado y seguir buenas prácticas de programación.
- Se debe incluir un archivo de prueba CSV con datos de ejemplo para demostrar la funcionalidad de carga masiva.

FECHA DE ENTREGA: x de octubre del 2026

5.4 Comentarios Generales
